Adaptive HACT: Semantics-Preserving Layouts for Verification Evidence

Da Xu Fei Wang

China Mobile Research Institute
Corresponding author: Da Xu, xudayj@chinamobile.com

October 2026 | Research manuscript v10.0

Abstract

Long-running software agents accumulate verification records whose layouts may change while their acceptance contract must remain fixed. Adaptive HACT separates a protected semantic registry and canonical checker outcomes from an adaptive certificate tree. Stable identities and local generation fencing preserve authorization through permutation and migration under explicit trust assumptions. Completion sets determine ancestor-packet refresh cost. We prove that even exact singleton and pairwise statistics cannot identify a globally optimal tree: two nine-check distributions have identical pairwise statistics but disjoint optimum sets under the implemented packet prices. An independent all-topology reference checks this witness. A fixed-order dynamic program provides an exact conditional reference; the practical compiler uses pairwise seriation and independent validation. Frozen clustered workloads show held-out padded-byte reductions of 23.43%, 10.75%, and 5.56%; independent and global controls show essentially no benefit. Re-encoding twelve archived candidates reduces compressed hierarchical streams by 4.36–41.84%. These are within-format results. Independently maintained tree and indexed-ledger services agree with an archived-record oracle in all 168 comparisons. Their answer bytes are equal by construction under common deterministic encoding, so this protocol cannot establish a bandwidth advantage. Local timings remain descriptive, and the archived ablation demonstrates no beyond-pairwise algorithmic gain. The results support semantics-preserving layout adaptation and its packet-cost model; comparative diagnostic utility and agent-productivity benefits require further evidence.

Keywords: agentic software engineering; verification evidence; runtime assurance; completion hyperedges; adaptive data structures; evidence virtualization; long-horizon agents.

1 Introduction

Software-engineering agents have progressed from single-shot code generation to repository navigation, tool use, test execution, and multi-step repair. SWE-bench made repository-level issue resolution measurable [18]; SWE-agent, OpenHands, and Agentless subsequently demonstrated how strongly agent interfaces, orchestration, and disciplined non-agentic baselines affect observed capability [23, 25, 24]. More recent long-horizon evaluation such as SWE-EVO exposes a harder regime in which tasks span many files, repeated revisions, and substantially longer trajectories [26]. In that regime, generation is only one part of the engineering problem: a system must also preserve the evidence that justifies accepting intermediate and final actions.

Verification-first approaches are therefore becoming central to agentic software engineering. Recent work treats verifiability as a first-class system property and instruments agent traces with contracts, replay, fault injection, and governance [27, 28, 29]. These directions address what should be verified and how an agent should be constrained. This paper asks a complementary systems question:

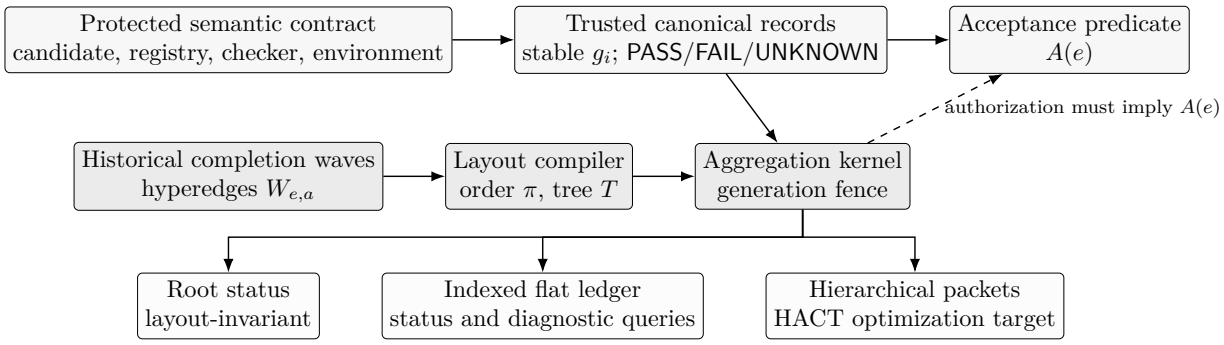


Figure 1: Adaptive HACT factors verification semantics from evidence representation. A valid layout may change aggregation cost and diagnostic hierarchy, but it cannot change the approved check set or the acceptance predicate. Root status and indexed-ledger queries need less structure; hierarchical packet generation is an additional format requirement.

> *Once a verification contract exists, how can a long-running agent system reorganize, migrate, and deliver the resulting evidence efficiently without giving the adaptive representation any authority to weaken that contract?*

A final decision may be one bit, but operators and downstream agents may also need bounded diagnostic slices, explicit coverage of declared obligations, and replayable records. Either an indexed ledger or a hierarchy can supply such queries. Learning a cheaper hierarchy is a representation problem; demonstrating that a hierarchy is preferable to an indexed ledger is a separate service-comparison problem.

We call the required separation **verification-evidence virtualization**. Stable semantic identities, candidate/checker/environment bindings, and canonical outcomes form a protected plane. An adaptive layout is a replaceable view over those identities. Migration changes a generation number and invalidates layout-specific publication tickets; the acceptance predicate is computed from canonical records, never from tree shape.

Adaptive HACT (Hyperedge-Aware Certificate Trees) is one realization of this architecture. Its second key idea is that evidence cost is driven by *sets* of checks that become final together. Pairwise similarity is easy to estimate, but it can lose exactly the higher-order structure that determines whether a subtree must be refreshed. HACT therefore treats observed completion waves as hyperedges and scores candidate layouts against those sets directly. Pairwise affinity is only a proposal mechanism.

Deployment requires separating representation cost from verification cost. A consumer that needs only a verdict can use a root projection; an indexed ledger can provide final statuses and diagnostic records. HACT optimizes an explicitly required hierarchical packet format. The current query study does not establish that this format is preferable to an indexed ledger. We therefore report within-format savings separately from the cost of introducing stronger verification, and price the conditions needed for break-even.

Evidence-layout independence. The paper formalizes the invariant sketched in Figure 1. A semantic registry and candidate identity are authoritative. Layouts are advisory data structures over those identities. A generation change invalidates layout-specific publication tickets and forces reconstruction from canonical trusted records. Under the stated trust assumptions, Theorem 6 shows that every authorized layout yields the same acceptance decision. This makes representation learning compatible with a fail-closed gate.

Why completion hyperedges matter. Evidence cost is driven by refresh events: when a set of checks becomes final, every ancestor packet intersecting that set may need to be re-emitted.

Marginal activation rates lose joint structure, and pairwise co-activity can still miss the probability that a larger subtree is touched. We therefore treat each completion wave as a hyperedge and score candidate layouts on the observed sets directly. Pairwise affinity is used only to propose an order; it is not substituted for the actual full-wave objective.

Where HACT is useful. Consumers may require a verdict, per-check state, or complete diagnostics. A root-status projection supplies the verdict; an indexed flat ledger can supply both state and diagnostic queries. HACT optimizes the third format when that format is externally required; diagnostic service requirements alone do not establish that a hierarchy is needed. This service distinction prevents a common accounting error—crediting evidence compression with the unrelated cost of introducing a stricter verifier.

Contributions. The paper makes three scoped contributions:

1. A semantic/layout contract and identity-stable, generation-fenced local construction, with a conditional authorization invariant through migration.
2. A completion-set packet-cost model, a fixed-order exact reference, and a practical pairwise-seriation compiler with validation against the incumbent. An all-topology separation establishes that pairwise statistics cannot in general identify an optimum; it does not establish a practical beyond-pairwise algorithmic advantage.
3. An auditable evaluation separating within-format byte savings from query-service costs, with independent reference answers, actual cache queries, strict paging, and separate resource accounting against an indexed flat ledger.

The ingredients include established clustering, tree optimization, caching and compression. The systems contribution is their explicit composition under an immutable acceptance contract and a joint-completion cost model, rather than a new general-purpose authenticated data structure or learning rate.

Scope. The experiments measure verification-evidence representation and local integration. They do not measure live-LLM coding quality, physical-WAN performance, or provider token billing. Those boundaries are collected in Section 13 instead of being repeatedly mixed into each positive result.

Roadmap. Section 3 defines semantics and the cost model. Section 4 develops the practical and exact-reference compilers; Section 5 prices migration; Section 6 proves layout-independent local publication soundness. Sections 7 and 8 describe the implementation and service-matched accounting. Section 9 maps the research questions. Section 10 and the strong-baseline audit report the frozen representation studies; Section 12 reports the corrected query study and ablation interpretation; Section 14 states the deployment implication.

2 Related Work and Positioning

Agentic software engineering and long-horizon verification. SWE-bench and subsequent software-agent systems made repository-level repair measurable [18, 23, 25, 24]; SWE-EVO extends the problem toward longer, multi-file software evolution [26]. Verification-driven and verification-first AI engineering argue that scalable checks, replay, and governance must be designed into the system rather than appended after generation [27, 28, 29]. HACT does not compete on solve rate. It addresses the evidence substrate required once an orchestrator already performs rigorous verification over a long horizon.

Table 1: Mechanism comparison with Huffman–Merkle Trees (HMT) [7]. The security and query services differ; no direct performance superiority is claimed.

Aspect	HMT	Adaptive HACT
Authoritative content	Committed mutable state; membership proofs	Declared checks and trusted local outcomes
Workload signal	Item access frequencies and their changes	Sets of checks completed between refreshes
Priced work	Hashing and access-weighted proof size	Ancestor-packet updates, cold builds and layout installation
Adaptation	Batched updates and tier migration	Permutation/tree changes with local epoch and generation fences
Evidence boundary	Authenticated data structure	Local record binding; no attestation or membership-proof service

Testing, impact analysis, and incremental computation. Test prioritization seeks earlier fault exposure [1]; regression-test selection uses change dependencies [2, 3]. The earlier IC-HACT ranker uses controlled mutation outcomes to prioritize checks, an instance of established fault-exposure methods. It is held fixed in the new source comparisons. Self-adjusting computation and build-system research already formalize dependency-aware recomputation [4, 5]. In contrast to test selection, our strict pass requires the entire approved registry on the identified candidate. We do not reproduce NameRTS or Ekstazi and claim no superiority over them.

Seriation and authenticated layouts. Spectral seriation orders items using an affinity matrix [14]. Average-linkage clustering likewise provides an established proximity-based ordering. Generalized alphabetic-tree optimization supplies the subtree/forest recurrence used here [6]. Workload-adaptive authenticated structures, including Huffman–Merkle Trees [7], already optimize layouts under access patterns. Our price counts repeated completion-frontier updates, including strict-epoch construction, under a fixed packet cap. Pairwise affinity proposes an order; full hyperedge observations score it. No global optimality or unproved NP-hardness result is claimed for unrestricted joint ordering and tree synthesis.

Hypergraph communication objectives. Balanced hypergraph partitioning already models communication using cut hyperedges and the number of parts they intersect; hierarchical variants capture communication across multiple memory levels [8]. HACT's per-node completion-set hits are another explicitly priced hierarchy objective. Its free tree shapes, packet arities and absence of balance constraints differ from those partitioning formulations. Their hardness results therefore do not establish hardness for HACT. The model-specific separation below establishes an information limit of pairwise summaries, rather than priority for the general use of hyperedges.

Online decisions with switching costs. Fixed-share learning tracks changing experts [15]; dependent sampling reduces switches compared with independent weighted-majority draws [16]. Full-information and bandit switching-cost settings have different achievable regret guarantees [17]. We use the full-information setting because one recorded completion wave can be scored against every frozen certificate layout without reexecuting the tests. This is not full counterfactual feedback for arbitrary agent strategies: if a layout changes the tests' execution schedule or model behavior, our exogenous-cost assumption may fail. Our bound specializes established tools to measured layout-installation costs; it is not a new minimax online-learning result.

Agent runtime assurance and evaluation. Proof-Carrying Agent Actions [9] and verification-portfolio commit gates [10] address related action-governance problems. Our local certificates are

neither signatures nor zero-knowledge proofs. SWE-bench evaluates real repository issues and patches [18]; our source mutations and scripted I/O repairs must not be called SWE-bench results. The current integration measures real orchestration and verification, not model reasoning quality.

Logging and serialization controls. Telemetry systems already batch and compress records [19]. HACT is not a replacement for those techniques: fewer padded bytes need not imply fewer compressed bytes. We measure both. Moreover, flat result ledgers and a trusted root-status endpoint can be much smaller than an archive of all intermediate aggregation updates; they satisfy different delivery needs and must not be hidden from the comparison.

3 Model and Trust Boundary

3.1 Semantic registry, execution, and display

Let $\mathcal{R} = (g_0, \ldots, g_{n-1})$ be a nonempty approved registry of unique check IDs. The candidate identity is

$$e = (\text{epoch}, \text{snapshot}, \text{checker}, \text{environment}, \text{registry}).$$

The snapshot covers source and protected acceptance inputs. The checker and environment are explicitly bound; an epoch increases even after an ABA return to identical source bytes. Only a trusted runner submits final PASS, FAIL, or UNKNOWN records. Missing, stale, skipped, incompatible, or timed-out outcomes are not passes.

An execution permutation σ_e determines when checks run. A layout $L = (\pi, T)$ independently maps tree positions to semantic registry indices. For a node covering positional interval $[i, j]$, its semantic coverage is

$$G_L(i, j) = \{g_{\pi(i)}, \ldots, g_{\pi(j)}\}. \tag{1}$$

The stored layout must contain an exact permutation, and every internal node's children must partition its positional interval without gaps or overlaps. The task graph is outside this mathematical tree; one worker can discharge many checks, and one integration check can cover several workers.

The acceptance predicate is independent of both permutations:

$$A(e) = \mathbf{1}\{\text{every } g_i \in \mathcal{R} \text{ has trusted, matching, current PASS evidence for } e\}. \tag{2}$$

An observed fail permits rejection before all checks finish. It does not assert coverage of the unexecuted remainder. Acceptance is relative to the approved predicates, not proof of all program behavior.

Definition 1 (Verification-evidence virtualization)**.** Fix a candidate identity e, semantic registry $\mathcal{R}$, and canonical trusted record set E. A verifier is *verification-evidence virtualized* if valid layouts over the same registry may change representation cost, retrieval structure, and packet timing but cannot change the semantic projection of E, and no layout can authorize unless $A(e)$ holds. We use *evidence-layout independence* for the corresponding authorization invariant.

Definition 1 is deliberately local to the declared verification contract. It does not claim that the contract itself is complete or that the checker is uncompromised; those are trust assumptions stated explicitly in Theorem 6.

3.2 Packet and lifecycle costs

Each internal node of arity d emits a fixed-width packet

$$c(d) = H + Sd, \qquad 2 \le d \le b = \lfloor (C - H)/S \rfloor . \tag{3}$$

Here $H = 512$, $S = 320$, and $C = 3072$ bytes, so $b = 8$. A mandatory field that exceeds its slot is rejected, not silently truncated. The cap applies to one certificate packet, not to all project memory, an entire registry manifest, or every LLM prompt. A manifest is a referenced control-plane artifact; its measured size is charged on migration.

Let $W_{e,1}, \dots, W_{e,q_e}$ be the semantic IDs whose final records become available between successive refreshes. For a strict new candidate epoch, the observed cost is

$$\ell_e(L) = W(L) + \sum_{a=1}^{q_e} \sum_{v \in \mathrm{int}(T)} c(d_v) \mathbf{1}\{W_{e,a} \cap G_L(v) \neq \varnothing\}, \quad W(L) = \sum_{v} c(d_v). \tag{4}$$

The first term constructs the full initially unknown tree. The CLI batches an executed prefix into groups of eight, flushing its final partial group. A successful strict epoch runs the whole registry. This conservative implementation does not reuse a pass across distinct source snapshots.

We call each $W_{e,a}$ a *completion hyperedge*: an arbitrary set of semantic obligations that becomes final between two refreshes. Equation (4) consumes these sets directly through interval-hit events. Pairwise statistics may propose a permutation, but the scored objective remains higher-order.

We separately study persistent-state refresh traces that omit $W(L)$ at each tick, as in a cache with correctly specified incremental dependencies. Each topology installation is still cold and explicitly priced. These traces evaluate the controller's cost model; they are not the strict candidate-gate timing experiment. They do not demonstrate safe dependency inference from arbitrary code.

3.3 Why more than invalidation or pairwise similarity is needed

The inherited HACT result shows that marginal check activation cannot identify the best aggregation tree under joint invalidation (Appendix C). IC-HACT further shows that even the full initial invalidation distribution cannot identify optimal progressive verification: early rejection and completion batching change the update sequence. Both distinctions remain relevant after allowing permutations.

Proposition 2 (Pairwise affinity is insufficient for full-wave scoring)**.** *There exist two distributions with identical singleton and pairwise co-activity probabilities, hence identical Jaccard affinities, but different activation probabilities for the same three-check subtree.*

Proof. On IDs $\{0,1,2\}$, let P be uniform on $\emptyset, \{0,1\}, \{0,2\}, \{1,2\}$ and Q uniform on $\{0\}, \{1\}, \{2\}, \{0,1,2\}$. Every singleton has probability $1/2$, every pair has joint probability $1/4$, and its Jaccard affinity is $1/3$. Yet the probability that the triple is hit is $3/4$ under P and 1 under Q. Thus a packet emitted on that interval has different expected cost. □

This elementary parity construction is not a new fact about higher-order statistics. The following extension establishes the stronger optimization consequence under the packet prices actually used by the artifact.

Theorem 3 (Pairwise summaries do not identify an optimal tree)**.** *For $H = 512$, $S = 320$ and maximum arity eight, there are two completion-set distributions on nine checks with identical singleton and pairwise probabilities but disjoint globally optimal tree sets. This holds over all valid leaf permutations and tree shapes for the persistent refresh cost, with any nonnegative multiple of full construction cost added.*

Proof. Use the parity distributions P, Q in Proposition 2 on $\{0,1,2\}$ and include six additional checks in every completion set. All nine singleton probabilities and all pairwise joint probabilities agree. Every subtree containing an additional check is hit with probability one. An internal subtree containing only parity checks is hit with probability at least $3/4$.

Let I be the number of internal nodes. Since the sum of internal arities is $9 + I - 1$, its full construction cost is

$$W(T) = 512I + 320(9 + I - 1) = 832I + 2560.$$

The arity cap requires $I \geq 2$. At most two internal nodes can contain only the three parity checks; their combined packet cost is at most 2304 (two binary nodes). Consequently expected refresh cost is at least $W(T) - 2304/4$. For $I \geq 3$ this is at least 4480 bytes, and construction cost is at least 5056 bytes.

With $I = 2$, the tree consists of the root and a single internal child, and $W(T) = 4224$. A child containing an additional check gives expected refresh cost 4224. A two-check parity child gives $4224 - 1152/4 = 3936$ under either distribution. A three-check parity child gives $4224 - 1472/4 = 3856$ under P and 4224 under Q. Thus P uniquely groups the triple, whereas Q groups one of its three pairs. No other shape improves these values. Their construction costs agree and are smaller than those of all $I \geq 3$ trees, so adding any nonnegative construction multiplier preserves the separation. □

Table 2: Analytical witness, expected refresh bytes. The all-topology optima are bold. Construction adds the same 4,224 bytes to both displayed shapes.

Only internal child	P	Q
Parity triple $\{0, 1, 2\}$	**3,856**	4,224
Any parity pair	3,936	**3,936**

A rule that sees only these pairwise probabilities receives identical input under P and Q and therefore cannot always identify an optimum. This is an identifiability result, not a claim about a particular clustering implementation or about typical software workloads. It uses persistent refresh events; it does not assert that a partially observed event is a complete strict verification epoch.

The independent reference in `experiments_v10/exact_separation.py` uses integer probability numerators and a subset recurrence over all root set partitions. It solves 511 nonempty subsets and considers 115,462 feasible root partitions per distribution and construction case. It finds one unordered optimum under P and three under Q. These counts treat sibling orders as equivalent. Separate literal tree enumeration checks the reference on smaller instances. This analytical witness is distinct from the frozen nine-check experiment, which enumerates permutations of one balanced shape.

4 Learning Layouts Without Changing the Contract

4.1 Practical catalogue and full-wave selection

Training waves define counts N_i, N_{ij} and affinity

$$a_{ij} = \frac{N_{ij}}{N_i + N_j - N_{ij}}, \tag{5}$$

with zero for an unobserved union and zero diagonal. The practical proposal uses established average-linkage clustering: join the closest current clusters, choose endpoint orientation by cross-boundary affinity, and break ties canonically. All semantic IDs, including disconnected and unseen checks, remain included exactly once.

Given the learned order, group consecutive leaves into blocks of at most eight, promote a singleton unchanged, and repeat upward. This bottom-up compact balanced construction does

not call a tree optimizer. The catalogue retains the incumbent first, includes original-order fixed8 when distinct, and proposes learned-order fixed8. Identical layouts are deduplicated. A separate validation cohort scores complete completion waves, including initial construction; ties retain the incumbent. The API returns an advisory choice, not deployment authorization. The registry and packet cap cannot change during selection.

For any proposal π, training episodes supply interval weights

$$r_\pi(i,j) = 1 + \frac{1}{m}\sum_{e=1}^{m}\sum_{a}\mathbf{1}\{W_{e,a} \cap G_L(i,j) \neq \varnothing\}. \tag{6}$$

These weights also define the exact-DP reference below. Pairwise affinity *proposes* an order; it does not replace the full-wave cost. The practical catalogue contains at most three candidates and uses no DP. Tree construction is $O(n)$ after ordering, not a linear-time claim for affinity construction, clustering or validation scoring. The reference implementation retains spectral ordering and exact fitting for research comparisons; neither is invoked by the practical default.

4.2 Exact conditional tree synthesis

Let $D(i,j)$ be the optimal subtree cost and $F_k(i,j)$ the optimal cost of an ordered forest with k nonempty children. With infeasible states equal to $+\infty$,

$$D(i,i) = 0, \tag{7}$$
$$F_1(i,j) = D(i,j), \tag{8}$$
$$F_k(i,j) = \min_{i+k-2\le u<j}\{F_{k-1}(i,u) + D(u+1,j)\}, \tag{9}$$
$$D(i,j) = \min_{2\le k\le \min(b,j-i+1)}\{r_\pi(i,j)c(k) + F_k(i,j)\}. \tag{10}$$

Theorem 4 (Exactness conditional on order)**.** *For a fixed π, nonnegative interval weights, and the packet model in Equation* (3)*, Equation* (10) *returns a minimizing feasible tree in $O(bn^3)$ time and $O(bn^2)$ space.*

Proof. Every feasible root has an arity and a contiguous partition of its interval. Its local contribution depends only on that interval and arity; all remaining contributions belong to independent child subtrees. The forest recurrence enumerates every such partition. Induction on interval length proves exactness. There are $O(bn^2)$ states, each scanning at most n splits. The apparent $F_1 = D$ dependency is used only on shorter intervals when constructing a forest with at least two components. □

This is a specialization of classical alphabetic-tree optimization [6]. The historical three-proposal reference multiplies compilation time by a constant, not by $n!$. Dense spectral computation can itself cost $O(n^3)$; the reference clustering implementation uses $O(n^2 \log n)$ heap operations and $O(n^2)$ storage.

Proposition 5 (Validation after learned ordering)**.** *Condition on a training set that fixes K complete layouts. For m independent identically distributed validation epochs with each layout cost in $[0,B]$, let $\widehat{L}$ minimize empirical validation cost. With probability at least $1-\delta$,*

$$J(\widehat{L}) \le \min_{L\in\mathcal{C}} J(L) + 2B\sqrt{\frac{\log(2K/\delta)}{2m}}. \tag{11}$$

Proof. Conditionally, the catalogue is fixed. Hoeffding's inequality [11] and a union bound over K layouts bound every empirical mean error by the displayed radius without its factor two. Inserting empirical costs on either side of the empirical minimizer yields the result. □

The guarantee is only relative to the fixed catalogue. It neither justifies choosing among learned permutations on their training errors alone nor covers distribution shift. We do not report this potentially loose bound as an empirical confidence interval; reported intervals use independent experimental seeds.

4.3 Compiler identity and scope

The practical default has no hidden microtree optimization. Exact DP is a reference and optional offline candidate. The block-backbone proposition and implementation remain in Appendix B solely as an ablation; its restricted-class exactness does not override the observed simpler-baseline advantage. At 1,024 checks, order learning still takes seconds in our measurement, so layouts should be cached and compiled away from the agent's per-action path. Validation of padded costs does not optimize compressed archives or model tokenization; adoption must measure the actual delivery representation.

5 Practical Online Selection: Price the Window, Not the Promise

Freeze a compiled catalogue $\mathcal{C} = \{L_1, \dots, L_K\}$. At event t, choose a layout before observing the current completion wave; afterwards score that same wave under every catalogue member. This is full-information feedback only because the check schedule is held fixed. It is not counterfactual feedback for arbitrary agent behavior.

A different layout is installed by serializing its canonical manifest and reconstructing its complete certificate tree:

$$M_{ij} = \begin{cases} 0, & L_i = L_j, \\ |\operatorname{manifest}(L_j)| + W(L_j), & L_i \neq L_j. \end{cases} \qquad \mathcal{L}_T = \sum_t \ell_t(A_t) + \sum_{t=2}^{T} M_{A_{t-1},A_t}. \tag{12}$$

The initial installation is separate. These byte prices match the evaluated uncompressed migration protocol. A deployment using compression must measure its own priced wire representation; the DP objective is not automatically an optimum for a compressed log.

The primary practical rule. Let $\bar{\ell}_{t,i}$ be the mean cost on the last $\min(w, t-1)$ observed events, and $j_t = \arg\min_i \bar{\ell}_{t,i}$ with deterministic ties. Starting from the calibrated incumbent a, use the frozen v3 rule

$$A_t = \begin{cases} j_t, & w(\bar{\ell}_{t,a} - \bar{\ell}_{t,j_t}) > M_{a,j_t}, \\ a, & \text{otherwise}, \end{cases} \qquad w = 64. \tag{13}$$

With no history, keep a. A tie in estimated savings does not trigger migration. During warm-up, the forecast horizon remains w even if fewer events were observed, matching the original baseline. This is an empirical break-even forecast, not proof that past savings repay a future switch. It can lose under abrupt reversal; preserving the frozen layout is always a legitimate alternative.

The runtime API enforces `choose()` before `observe()`, bounds history to 64 vectors, rejects nonfinite costs, and checkpoints only at event boundaries. Replaying all 80 archived v3 streams, with periodic checkpoint/resume, must reproduce the original baseline's full action and byte-cost paths exactly. The exercise validates implementation equivalence, not a new performance dataset.

Why fixed-share is no longer primary. On the archived stationary, long-shift and rapid-shift streams, the coupled fixed-share policy is more expensive than the priced window. Its standard tracking analysis appears in Appendix A, with the negative results retained. Neither the appendix bound nor this empirical comparison establishes general superiority over other switching-cost policies.

Algorithm 1 Deployable evidence control; inference and checking remain separate

Require: Protected registry, checker and snapshot identities; frozen layout catalogue
for each event **do**
  Choose from past evidence using Equation (13), or keep the frozen layout
  if layout changes **then**
    Revoke tickets, increment generation, remap stable IDs, rebuild and price the installation
  end if
  Receive trusted completion records for the fixed check schedule
  Reject stale, conflicting or malformed records; aggregate bounded packets
  Publish only the complete current contract result; otherwise retain FAIL/UNKNOWN
  Expose a root-status card; retrieve diagnostic packets only when needed
  Score the observed event and append its cost vector to the bounded window
end for

6 Stable Evidence Through Permutation and Migration

A result token carries (epoch, g_i, b_i), where b_i binds the semantic ID to the registry, source snapshot, checker, and environment. It does not carry a display position as its identity. A layout-specific kernel maps stable IDs to current positions. During migration it rebuilds from canonical trusted records; it never transplants an ordinal cache entry into a different permutation.

Publication tickets additionally carry a monotone layout generation and layout hash. A layout switch revokes the issued ticket, advances the generation, and forces reconstruction. Returning to an earlier tree therefore cannot revive that tree's old ticket. An in-flight token may remain valid through a layout-only change, but a new candidate epoch fences it. Duplicate identical evidence is idempotent. Conflicting final records in one epoch revoke authorization: a recorded fail remains a fail; otherwise the obligation becomes unknown. Resolution requires a new epoch, not a more persuasive agent summary.

Theorem 6 (Layout-independent local publication soundness)**.** *Assume complete approved predicates and dependency identities, a trusted checker executing immutable identified inputs, collision-resistant binding, and atomic local epoch/generation checks. For any sequence of valid layout permutations, any advisory learning outputs, and any delivered evidence order, authorization implies Equation* (2)*. Moreover, two valid layouts reconstructed from the same canonical record set have identical root pass/fail/unknown counts and therefore the same semantic status projection. If each checker is sound for its declared predicate, the declared predicates hold on the authorized candidate.*

Proof. A canonical pass requires a current token for the same semantic ID and immutable identities. Permutation is a bijection, so rebuilding moves each such record to exactly one position without changing its meaning. Exact child partitioning preserves coverage and pass/fail/unknown counts by induction up the tree. The root can be authorized only with $(n, 0, 0)$ and matching current registry, epoch, generation, and layout. Old tickets fail those checks. Conflicts prevent authorization. No choice made by the learner writes trusted outcomes or changes the registry. □

This is a conditional local guarantee, not remote attestation or distributed consensus. An incomplete suite, compromised checker, omitted dependency, or nondeterministic external service can invalidate a broader correctness claim. The CLI's file copy is not an OS sandbox. An external merge/deployment service must atomically compare the verified candidate and integration-head identities at commit; a report file alone is not that transaction.

Table 3: Separate ledgers avoid converting byte savings into unsupported latency or token claims.

Measurement	How obtained	Included in optimized objective?
Certificate bytes	Actual padded packets; independently reconstructed from waves	Yes
Migration bytes	Canonical layout manifest plus forced full rebuild	Yes, online traces
Fit and scoring time	Monotonic-clock measurements	Reported separately
Preparation, copying, checking, aggregation	Wrapper timers and outer call duration	Reported separately
Active harness episode	Supervisor startup through task completion and shutdown	Reported separately
Prompt bytes and construction time	Captured actual HTTP messages and context builder	Reported; not tokens
Hosted token usage and API billing	No provider inference was run	Unknown, never zero-imputed

7 Implementation and Cost Accounting

The v5 artifact adds `practical.py`, `deployment.py`, `token_budget.py` and `tls_probe.py`. The practical CLI produces a balanced catalogue without calling either exact or block DP. The historical exact and block compilers remain separately callable. The existing stable-ID guard, priced window, codec and Future Code bridge are retained. The strict CLI compares protected input *sets* and hashes, detecting added tests or configuration files as well as changed and deleted ones. Schema-2 contracts bind the approved checker and environment and refuse silent reuse of legacy contracts. Source manifests are checked before and after snapshotting and verification; mutation of an observed snapshot prevents a pass.

The bridge runs as an ordinary subprocess acceptance gate in the unmodified Future Code v1.2 runtime. The supervisor still owns worker leases, private attempts, bounded contexts, and serialized integration. Non-pass exits cause candidate rejection. Detailed evidence and binary certificate packets live outside candidate write scopes; only a bounded JSON result card returns to the worker. The plug-in accepts a precompiled layout. The online selector and migration guard are separately integrated and stress-tested as a library path; Section 13 states the remaining end-to-end deployment gaps.

A different scalar objective could combine these quantities only with declared unit conversions and observable counterfactual costs. This implementation does not optimize unknown model latency or billing. The exact-byte claim also depends on the stated padded format; compression defines a different, potentially nonseparable objective for which this DP is not claimed exact.

8 Service-Matched Delivery: What HACT Can Save

8.1 Consumer requirements and delivery formats

The local aggregator receives bounded packets; a logging collector may receive their complete stream; a supervisor normally receives a status projection and retrieves details selectively. Define $\psi(e, c)$ as canonical status text containing semantic identity, counts and local authorization, but no display positions. For the same trusted semantic report,

$$\psi_{L_1}(e, c) = \psi_{L_2}(e, c). \tag{14}$$

Its byte length and token count under any fixed deterministic tokenizer are therefore identical. This is a useful negative control: layout optimization cannot shrink a layout-independent root input.

An indexed flat ledger can provide final per-check state and diagnostic queries without preserving the complete intermediate hierarchy. HACT's layout objective applies when the hierarchical diagnostic stream itself is part of the required service.

Proposition 7 (Common encoding fixes answer bytes)**.** *If two services produce the same canonically ordered semantic answer and apply the same deterministic paging, framing and compression with identical parameters and reset boundaries, their complete application byte streams are identical, regardless of their internal indexes.*

Proof. Each stage is the same deterministic function of identical input. Equality therefore propagates through every stage to the transmitted bytes. □

The corrected query protocol has these conditions. Its equal-byte result checks conformance; it cannot test a bandwidth superiority hypothesis. Internal indexing can still change setup, update and aggregation costs. Testing a bandwidth advantage requires a different declared wire protocol, such as client-held state with delta delivery, while preserving identical client-visible answers and charging both clients' state and update traffic. Such a protocol is not evaluated here.

Diagnostics preserve full header/card records and page whole packets under a 16,384-byte limit. A packet that cannot fit is rejected rather than truncated. For a real supplied tokenizer τ and explicitly reserved framing budget r, token-aware admission additionally requires

$$|p|_{\mathrm{UTF8}} \le C_{\mathrm{bytes}}, \qquad |\tau(\mathrm{decode}_{\mathrm{UTF8}}(p))| + r \le C_{\mathrm{tokens}}. \tag{15}$$

The implementation retokenizes each complete proposed page, including its prefix; it does not assume additivity of fragment token counts. The optional measurement command requires an actual `tiktoken` encoding [20]. Provider framing and billed usage remain properties of a provider adapter, not of the byte objective.

8.2 Codec boundaries are part of the service

Canonical unpadded JSON, independently compressed packets, and whole-epoch DEFLATE preserve the same logical records but expose different streaming and compression behavior. The padded objective is therefore not claimed to optimize compressed bytes or model tokens exactly. Practical selection is advisory until the deployed representation is measured with the deployed codec.

This distinction also separates representation choice from verification choice. If a deployment already requires protected hierarchical verification, fixed and learned layouts can be compared at marginal cost under the same service. If the baseline used ordinary testing with no protected evidence service, adopting the verifier is a separate engineering decision and its additional cost cannot be charged to, or justified by, layout compression alone.

8.3 Break-even accounting

Consider q copies of the *same* workload and delivery service through one nonoverlapped bottleneck. Let B_0, B_1 be codec-matched application bytes per copy, $R > 0$ effective application goodput, $E \ge 0$ additional encoding/decoding work per copy, and $C \ge 0$ consistently amortized one-time incremental work. Common setup cancels only when it is actually common. The conditional net benefit is

$$d = (B_0 - B_1)/R - E, \qquad \Delta(q) = qd - C. \tag{16}$$

A strictly profitable integer copy count exists only when $d > 0$, and then

$$q_{\min} = \lfloor C/d \rfloor + 1. \tag{17}$$

A zero gain retains the baseline. The implementation uses decimal arithmetic at equality, rejects mismatched service IDs, and returns `UNKNOWN` when required measurements are absent. An explicit zero is accepted only as a declared assumption or observation.

When both variants use the same protected gate, C represents extra layout work rather than the entire gate cost. Repeating delivery can amortize preparation of the same candidate; it cannot amortize preparation of future candidates as though they were the same workload. Parallel links, computation overlap, cached diagnostics, queueing, and connection setup require direct critical-path measurement. RTT alone does not increase the byte-dependent term when both alternatives pay the same setup and request count.

For the common-codec query comparison, the byte term cancels. If incremental tree setup costs C seconds and the tree saves s measured seconds per query after matched update and codec work, break-even instead requires $qs > C$ with $s > 0$. These quantities must be measured over independent traces; the single local campaign does not estimate a deployment-wide s. A query issued after verification completes must exclude the archived checker time from its response-latency denominator. A candidate-start latency question includes that common work. The two estimands answer different operational questions.

8.4 Remote-measurement boundary

The artifact includes an opt-in TLS measurement kit with certificate validation, hostname checking, bounded frames, validated decompression, and acknowledgments that bind received length and digest. Local tests exercise cold and reused sessions, rejected hostnames, untrusted roots, malformed acknowledgments, and truncated records. These tests validate the measurement path; they are not WAN performance measurements.

A qualifying remote experiment must record host/path provenance, codec, connection policy, matched payloads, application goodput, and transport observations such as RTT or retransmissions where available, following the separation between application payload and transport behavior emphasized in TCP throughput testing [22]. TLS authenticates the configured server, not the truthfulness of a checker. Attestation, Byzantine consensus, client authorization, non-repudiation, and OS isolation remain outside HACT's local semantic guarantee.

9 Evaluation Design

Table 4: Research questions and evidence boundaries.

RQ	Question	Evidence
RQ1	When does ordering reduce hierarchical packet cost?	Frozen synthetic workloads with disjoint training, validation and held-out epochs
RQ2	What does practical compilation trade for simplicity, and is semantic output invariant?	Archived source re-encodings and local authorization checks
RQ3	Is a beyond-pairwise algorithmic benefit demonstrated?	Frozen ablation re-analysis; the separate all-topology witness establishes identifiability, not deployment benefit
RQ4	When can delivery savings repay extra work?	Explicit conditional break-even scenarios
RQ5	Do the services preserve answers, and what resources do they consume?	Independent oracle and separate resource ledgers; common encoding precludes a wire-byte difference

The v5/v7 records and the v8 ablation are archived measurements. The v9 query study is a new retrospective execution of corrected service code over the same source records, not new checker

trials or a preregistered prospective study. Its input files are digest-bound. Host elapsed timings are descriptive; network arrival values are explicitly modeled. Current result macros, query rows and signed ablation prose are generated by an independent arithmetic audit. The v10 all-topology witness is a deterministic check of an analytical construction, not another held-out workload or a new repair sample.

10 Representation-Cost Evaluation

10.1 Data and provenance

The frozen v5 contract fixes three registry sizes (128, 512, 1,024), three completion families (clustered, independent, global), and three seeds (55011–55013). Each of the 27 workloads contains 128 training epochs, 128 validation epochs, and 256 held-out epochs. Random draws are sequential disjoint samples; deterministic global observations naturally coincide. The catalogue is fit only on training data, selected on validation, and reported on held-out data. Cold strict-epoch construction is included in every objective. The nine size/family summaries report seed means and observed ranges; with three seeds per cell we do not turn episode counts into pseudo-replicated confidence claims.

The source study re-aggregates twelve archived candidates, four each from NetworkX, Toolz, and Future Code, under three layouts. All three representations consume the same trusted check records, execution sequence, and batches of eight. The 36 outputs are layout replays, not fresh checker executions or new autonomous repairs. Historical v3/v4 records remain in the artifact under their original versioned provenance; they are not merged into the v5 empirical counts.

10.2 RQ1: learned ordering helps only when completion structure exists

Table 5: Frozen synthetic study. Reduction is selected-layout held-out padded cost relative to original-order balanced fixed8; positive is better. Fit includes affinity/order learning and catalogue construction, not validation scoring. Ranges are over three seeds.

Checks	Family	Mean (%)	Range (%)	Fit median (s)	Changed
128	cluster	23.43	22.40–24.45	0.018	3/3
128	independent	0.00	0.00–0.00	0.016	0/3
128	global	0.00	0.00–0.00	0.032	0/3
512	cluster	10.75	10.06–11.13	0.523	3/3
512	independent	0.07	0.00–0.20	0.548	1/3
512	global	0.00	0.00–0.00	0.645	0/3
1,024	cluster	5.56	5.16–5.80	2.991	3/3
1,024	independent	0.00	0.00–0.00	3.086	1/3
1,024	global	0.00	0.00–0.00	3.462	0/3

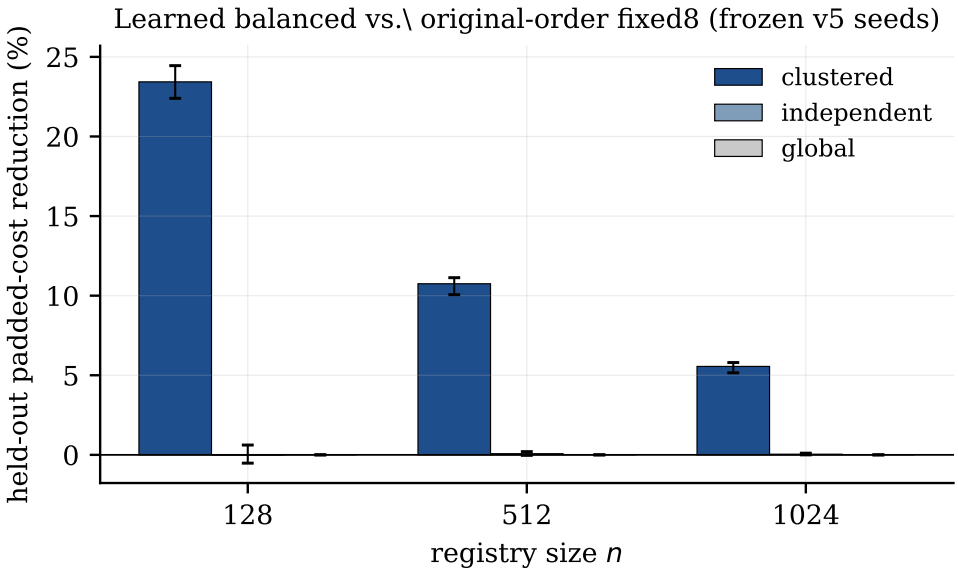


Figure 2: Held-out padded-cost reduction versus original-order fixed8. Marks show the seed mean and whiskers the observed three-seed range. All nine clustered seeds improve; the null controls show no systematic benefit.

Clustered completions yield mean held-out reductions of 23.43%, 10.75%, and 5.56% at 128, 512, and 1,024 checks. More importantly than the means, every clustered seed is positive: the observed ranges are 22.40–24.45%, 10.06–11.13%, and 5.16–5.80%. Global updates yield exactly zero improvement in all three sizes. Independent updates yield zero except for one 512-check seed, producing a 0.07% mean and a 0.00–0.20% range. The experiment therefore supports a conditional claim: ordering is useful when completion waves contain stable locality, and the validation stage can retain the incumbent when they do not.

The practical path invokes no tree DP. At 1,024 checks, catalogue fitting still takes about three seconds on the recorded host, so learned layouts belong off the agent's per-action path and should be cached. The decreasing percentage with registry size is consistent with substantial layout-independent cold construction in this fixture; it is not presented as a scaling law.

The first synthetic execution exceeded its tool time limit after persisting 26 complete records. No worker remained; explicit resume skipped those immutable records and completed the final frozen case without changing inputs, selection, or seeds. The interruption record remains part of the artifact provenance.

10.3 RQ2: compilation tradeoffs and semantic invariance

Table 6: Re-serialization of twelve archived source candidates. The first three columns compare learned balanced8 with original-order fixed8. The last column is the learned-balanced penalty relative to learned exact after whole-epoch compression.

Project	Padded saving (%)	Canonical saving (%)	Epoch saving (%)	vs. exact penalty (%)
networkx	48.63	48.74	41.84	11.68
toolz	8.78	8.70	4.36	1.52
future	29.12	29.23	18.53	6.67

Learned balanced layouts reduce whole-epoch compressed bytes by 41.84% for NetworkX [12], 4.36% for Toolz [13], and 18.53% for Future Code. Across all twelve candidates, batch-compressed application bytes fall from 416,046 for fixed8 to 299,081 for learned balanced. Learned exact uses 278,831 bytes. The balanced streams are 11.68%, 1.52%, and 6.67% larger than exact, respectively, with exact as the denominator. Practical compilation removes cubic tree synthesis from the default path at this observed representation-cost tradeoff. The DP is exact for its additive padded objective; these compressed sizes do not establish compressed-byte optimality.

Semantic counts, verdicts, and local authorization agree across all 36 replays. Root-status text is byte-identical in every layout triple and occupies 593–594 UTF-8 bytes, directly validating the negative control in Equation (14). Exhaustive diagnostic pages total 148 for fixed8, 95 for learned balanced, and 87 for learned exact, so the practical layout uses 35.81% fewer pages than fixed8. The largest page is 16,364 bytes under a 16,384-byte cap. These page counts describe an exhaustive diagnostic service; they do not imply that a model will request every page.

10.4 RQ4: break-even depends on the deployed service

The fixed-to-balanced whole-epoch compressed difference is 116,965 bytes per twelve-candidate cohort. Under the explicitly assumed goodput of 64 KiB/s and $E = 0$, Equation (16) converts that difference to 1.784744 seconds of gross delivery time per cohort copy. This is service-model arithmetic from measured bytes, not a measured WAN speedup.

Table 7: Conditional minimum profitable copies of the same twelve-candidate cohort for whole-epoch compression, assuming $E = 0$. Column headers are extra work per candidate; corresponding one-time cohort costs are 0.6, 6, and 60 seconds.

Assumed goodput	0.05 s	0.5 s	5 s
8 KiB/s	1	1	5
64 KiB/s	1	4	34
1 MiB/s	6	54	538

The table exposes the operating region rather than selecting a universal winner. At 64 KiB/s, for example, an assumed 60 seconds of extra cohort work requires 34 repetitions of the same delivery service before modeled savings become strictly positive. New candidates incur new preparation and cannot reuse that amortization. The artifact stores 27 scenarios spanning three codecs, three rates, and three extra-cost assumptions; negative outcomes are retained, and mismatched or incomplete services return `INCOMPARABLE` or `UNKNOWN` rather than an invented benefit.

10.5 Controls, software validation, and independent reconstruction

Historical controls remain useful only in their original roles. Earlier receiver-paced loopback experiments showed that compression itself can dominate layout choice, and earlier harness timings showed no end-to-end speedup from the strict wrapper. Because those measurements use different cohorts and services, this manuscript does not subtract them from the new replay data. The block compiler's historical 1.54–5.34% held-out penalty to same-order balanced8 is retained as the ablation that motivated the simpler default.

The v5 prototype passes 329 tests, including 76 new cases, while the unchanged Future Code suite passes 263 tests separately. New tests cover no-DP practical compilation, service mismatch, missing costs, exact break-even, whole-text token admission, TLS trust failures, stable-ID migration, and deliberate data tampering. A separate auditor imports no producer optimizer, selector, or codec and independently reconstructs initialization/wave costs for all 27 workloads, all 36 serialized replays, codec sizes, diagnostic pages, 27 planning scenarios, and 131 summary

Table 8: Additional held-out comparison against original-order balanced8. Mean percentage reduction over three seeds; positive is smaller. HACT is the original validation-selected learned layout. Random best is a hindsight-only diagnostic, never used for selection. No confidence interval is inferred from three seeds.

Checks	Family	HACT (%)	Frequency (%)	Random median (%)	Random oracle (%)
128	clustered	23.43	13.91	-0.23	1.39
512	clustered	10.75	5.03	-0.06	0.28
1024	clustered	5.56	1.96	0.00	0.07
128	independent	0.00	0.00	-0.04	0.22
512	independent	0.07	-0.04	0.03	0.11
1024	independent	0.00	0.03	0.01	0.08
128	global	0.00	0.00	0.00	0.00
512	global	0.00	0.00	0.00	0.00
1024	global	0.00	0.00	0.00	0.00

numerical comparisons. This is independent arithmetic and provenance reconstruction, not third-party certification.

11 Additional Strong-Baseline Audit (V7)

11.1 Contract: archived observations, new controls

To challenge the choice of baseline rather than improve a favorable benchmark by editorial selection, we added a pre-specified retrospective analysis against two additional permutation families and an explicit flat-ledger representation, performed on archived observations under a frozen rule. The inputs are the frozen v5 training/validation/held-out waves and the twelve previously executed checker traces. This audit introduces *no* fresh checker execution, live agent run, physical-WAN transfer, or measured provider-token usage. The program and tamper tests are in `experiments_v7`, `tests_v7`, and the machine-readable GitHub Actions artifact. The new control results must not be counted as independent project replications of the v5 runs.

Permutation controls. A frequency-order baseline sorts semantic IDs by the marginal number of training-wave occurrences, retaining canonical-ID tie breaks. The selected learned layout is fixed by the original separate validation cohort. Eight deterministic random permutations are generated using case-specific SHA-256-seeded randomness, then scored only on held-out epochs. Their median is a random-order control; their best result is an *unavailable-at-deployment hindsight oracle*, not a trained or proposed selector. Every candidate uses the same eight-ary tree, packet price, cold-construction cost, and held-out completion waves. The audit independently recomputes all published v5 held-out layout totals and aborts on a mismatch.

The learned layout wins against these simple permutation controls in every clustered size. At the 1,024-check independent control, the frequency ordering obtains a small positive improvement even though the validation-selected HACT layout retains its incumbent; we do not reselect on this held-out fact. The global updates provide no ordering advantage, as expected. These data strengthen the conclusion that *structured completion locality*, not arbitrary reordering, explains the primary synthetic effect.

11.2 Safety controls and prospective evaluation

An adversarial property test enumerates all 3^4 PASS/FAIL/UNKNOWN assignments across three permutations of a four-obligation registry. Migration invalidates old tickets; conflicting same-epoch evidence cannot be revived by changing the layout. These bounded checks complement the

conditional local invariant; they are not distributed-system verification.

The corrected service-equal study appears in Section 12. The separately published live-evaluation protocol remains unexecuted. No agent solve-rate, provider-token or physical-WAN conclusion follows from the archived source replays.

12 Diagnostic-Service Evaluation and Ablations

12.1 RQ5: corrected service-equal protocol

The diagnostic contract fixes semantic selections and answers independently of tree positions. Q1 reports full status, Q2 localizes failures, Q3 lists unfinished obligations, Q4 discloses coverage, Q5 replays a completed wave, and Q6a/Q6b restrict status to the frozen unit/integration predicates. Failure groups use stable test-file paths; these predicates and groupings do not assert a classification of test adequacy. Coverage includes the actual declared, executed and missing ID sets. Epoch is a completed-wave count; foreign snapshots are rejected and older snapshots are rebuilt consistently.

The tree service applies the learned permutation to kernel positions and uses maintained subtree capsules for status aggregation. The indexed ledger reads per-check events through its persistent index. Neither service calls the archived-record reference answer implementation. A separate oracle compares both answers to that reference; equality between the services alone is insufficient. All 168 comparisons and 84 layout-invariance checks agree. New regressions include a deliberately shared query error, which the independent oracle rejects, and older-snapshot queries after full completion.

Both services use bounded FIFO query caches, identical JSON-lines answer framing and DEFLATE level six. Every page descriptor counts inside its byte cap. Indivisible oversized fields are rejected; decoding verifies complete reconstruction. The query workload includes Q3, seeded additional replay waves, both original and learned layouts, and actual cold/warm query pairs. It comprises 364 paired query rows across 24 matched trajectory/layout traces. Each arm records 182 actual cache hits. Arm order is seeded separately. This is a single local timing campaign over the archived candidates, not evidence about independently selected live tasks.

12.2 Wire bytes, storage and time are separate ledgers

Table 9 reports only compressed complete-answer bytes. Each arm totals 422,976 bytes. Proposition 7 implies this equality before measurement. It is a conformance result, rather than evidence for or against a bandwidth advantage of hierarchical indexes. The previously declared strict requirement of both fewer bytes and lower modeled arrival time cannot be met under this common-codec protocol. A different wire protocol requires a separate comparison; index-maintenance and aggregation benefits require repeated measurements of those resources.

Table 9: Corrected query study. Original/learned layouts and cold/warm cache requests are included. Logical storage and update bytes are excluded from wire totals. "Benefit" requires strictly fewer bytes and lower modeled time.

Family	Rows	HACT wire (B)	Flat wire (B)	Benefit
Q1	48	68,124	68,124	no
Q2	36	23,592	23,592	no
Q3	48	68,072	68,072	no
Q4	48	128,676	128,676	no
Q5	88	47,544	47,544	no
Q6a	48	68,188	68,188	no
Q6b	48	18,780	18,780	no

Canonical serialized persistent-state totals are 2,279,524 bytes for HACT and 733,592 for flat across the matched traces. These exclude duplicate query-cache objects and describe neither resident RAM nor traffic. Local setup/maintenance elapsed totals are 0.1562 and 0.0267 seconds, respectively. Setup is recorded once per trace, then allocated uniformly across that trace only for the arrival model; it is not assigned solely to Q1. Logical update bytes remain a separate resource measure. These local timings support descriptive accounting rather than statistical claims about deployment latency.

Modeled arrival adds archived serial completion time, allocated setup, measured query aggregation/encoding/decoding, and compressed bytes divided by declared goodput. It excludes no setup stage and adds no storage footprint to wire bytes. All entries are marked arithmetic. Under this particular critical-path model, eliminating flat aggregation entirely could reduce arrival time by at most 0.14%. The observed-ratio estimate is reported separately from this infinite-acceleration ceiling. The common archived-checker term is relevant to arrival from candidate start; it must be removed when studying response latency for a query issued after checking has finished. Physical transfer, concurrency and production demand remain unmeasured.

12.3 RQ3: corrected interpretation of the frozen ablation

The archived ladder compares frequency order, pairwise cluster order, budgeted direct-wave refinement, original-order exact tree DP, and a small fixed-topology permutation reference. It is re-analyzed here without relabeling its rows as new experiments. The direct-wave search beats pairwise seriation in 0 of 9 clustered workloads. Frequency ordering has a mean cost penalty of 7.56% on the clustered subset and 2.53% across all workloads. Original-order exact DP has cost penalties of 15.17% and 5.12%, respectively, against learned-order balanced trees. This comparison changes both order and shape and therefore cannot bound same-order tree-shape headroom. At $n = 9$, the maximum fitted-sample gap is 5.93%, over permutations of the fixed compact-balanced topology only. It is neither a global optimum over tree shapes nor a held-out guarantee. These results do not establish a beyond-pairwise advantage for the deployed algorithm. The direct-wave refinement is a time-budgeted heuristic, so its negative outcome does not prove that no higher-order optimizer can improve the layout. A proper tree-shape ablation must hold the learned order fixed. Fixed-order training optimality also does not imply held-out optimality. Proposition 2 retains its information-separation meaning without an empirical claim that the deployed compiler exploits beyond-pairwise structure. Theorem 3 strengthens that analytical distinction to disjoint global optimum sets on a constructed instance. It does not alter the frozen ablation's empirical outcome.

12.4 Audit scope and correction record

The historical audit's seventy consistency entries passed despite errors in v8's prose. The revised independent path imports no service, optimizer or codec implementation; it verifies input digests and regenerates the current result macros, query table and signed ablation paragraph from committed JSON. Tests corrupt generated percentages, interpretation text and study rows to ensure rejection. The artifact retains the original v8 JSON and documents its withdrawn service-cost interpretation. In particular, the old optimum-gap percentage and DP-improvement interpretation are not used as current evidence. The original Amdahl arithmetic yields only 0.18% maximum speedup, with its own historical denominator. It is not combined with the corrected timing campaign. Functional query/codec tests complement this arithmetic audit; passing either is not third-party certification.

13 Implications and Threats to Validity

Operational implication. An indexed ledger is the appropriate initial baseline for the declared status, coverage and diagnostic queries. The corrected study's common codec precludes a wire-byte advantage at that service. If a deployment independently requires the hierarchical packet format, learned ordering can reduce its within-format cost on structured completion traces. This conditional saving does not establish the value of maintaining a hierarchy in the first place. Exact fitting remains an offline reference, and a frozen incumbent remains a valid deployment choice.

Construct and internal validity. The primary optimization target is an additive padded-packet model. Canonical bytes, compressed bytes, page counts, serialized state, RAM, model tokens and latency are distinct metrics. The corrected study separates those it measures, and its arrival model is explicitly conditional. Common checker time can mask post-query responsiveness, so the reported Amdahl fraction is not a bound on all diagnostic workloads. Query caches are exercised, but one local timing campaign provides no statistical deployment-speedup estimate. Older scripts and results are versioned historical records; corrected studies do not turn archived evidence into independent samples. The newly generated result prose addresses sign, unit and subset errors that the earlier consistency audit missed.

External validity. The source cohort contains twelve archived candidates from NetworkX, Toolz and Future Code. Synthetic cells have three seeds and fixed generators; source cases are not an independently selected natural-bug or demand corpus. Repeated layouts and cache requests do not increase the number of independent candidates. No new hosted model, physical-WAN or provider-token measurement is claimed. The paired query study is a retrospective implementation comparison, not a live long-horizon agent evaluation. New repositories and time periods, prospectively sampled query demand and repeated host measurements are needed to establish an applicability region beyond the packet-format results.

Security and semantic boundary. Theorem 6 assumes a complete approved predicate set, correctly bound dependencies, a trusted checker, collision-resistant binding and atomic local fencing. HACT provides no checker attestation, distributed consensus, non-repudiation or OS sandbox. Its local capsules are not authenticated membership proofs. A merge or deployment service must separately bind the verified candidate to the integration head atomically. Layout independence preserves a declared contract; it cannot establish the contract's adequacy.

Research priorities. The next study should compare independently implemented services under prospectively sampled diagnostic requests, including state changes, cache misses and migration, on held-out repositories. It should test whether any measured benefit repays compilation and maintenance under an explicit critical-path model. Live-agent and multi-host evaluation can follow when such a service advantage is established; their outcomes remain unknown.

14 Conclusion

Adaptive HACT supplies a conditional local mechanism for changing evidence layouts while preserving the acceptance contract. Completion sets define its packet-refresh objective. Identical pairwise statistics can require different globally optimal trees under this model, while the demonstrated practical algorithm uses pairwise seriation and independent validation. Frozen experiments support within-format byte and exhaustive-page reductions when completion locality is structured. The corrected service comparison gives equal answer bytes as a consequence of the common codec and records additional tree-state costs. It cannot establish a bandwidth advantage. Comparative

diagnostic utility and agent-productivity benefits remain unestablished. The supported design lesson is precise: representation may adapt, while semantic identities and the evidence authorizing progress remain protected.

Data and Code Availability

The source, tests, frozen observations, corrected query study and independent audits are available at
`https://github.com/xuda1979/future-code/tree/main/hact-paper/Adaptive_HACT_v5`.
The release manifest records file sizes and SHA-256 digests; it provides byte integrity, not a digital signature. Historical records remain versioned. The new analytical witness is in `results/v10`; it adds no source candidate, checker run, live-agent trial or deployment measurement. The accompanying submission-readiness report distinguishes completed checks from evidence still needed for the broader service claim.

Acknowledgments

OpenAI Codex was used to assist with the manuscript revision, including wording and claim calibration throughout, the proof and analytical witness in Theorem 3, and the associated reference code and tests. It also assisted with the corrected service implementation and numerical audit described in Section 12. The authors remain responsible for the final scientific content.

References

[1] Gregg Rothermel, Roland H. Untch, Chengyun Chu, and Mary Jean Harrold. Prioritizing test cases for regression testing. *IEEE Transactions on Software Engineering*, 27(10):929–948, 2001.
`https://doi.org/10.1109/32.962562`.

[2] Milos Gligoric, Lamyaa Eloussi, and Darko Marinov. Practical regression test selection with dynamic file dependencies. *Proceedings of ISSTA*, 2015.
`https://users.ece.utexas.edu/~gligoric/papers/GligoricETAL15Ekstazi.pdf`.

[3] You Wang, Michael Pradel, and Zhongxin Liu. Names Are All You Need: Effective and Safe Regression Test Selection for Python. arXiv:2605.25356v1, May 2026.
`https://arxiv.org/abs/2605.25356v1`.

[4] Umut Acar, Guy Blelloch, Matthias Blume, Robert Harper, and Kanat Tangwongsan. A Library for Self-Adjusting Computation. *Electronic Notes in Theoretical Computer Science*, 148:127–154, 2006.
`https://doi.org/10.1016/j.entcs.2005.11.043`.

[5] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build Systems à la Carte. *Proceedings of the ACM on Programming Languages*, 2(ICFP), Article 79, 2018.
`https://doi.org/10.1145/3236774`.

[6] Hirendu Vaishnav and Massoud Pedram. Alphabetic Trees: Enumeration and Optimization with Applications to VLSI Design. Undated author-hosted manuscript, Sections 4.1–4.4.
`https://www.mpedram.com/Papers/alp-trees-journal.pdf`.

[7] Ziheng Shangguan, Aviv Yaish, and Dahlia Malkhi. Authenticated Data Structures for Dynamic Workloads. arXiv:2608.25206v1, August 2026.
`https://arxiv.org/abs/2608.25206v1`.

[8] Pál András Papp, Georg Anegg, and A. N. Yzelman. Partitioning Hypergraphs is Hard: Models, Inapproximability, and Applications. *Proceedings of SPAA*, 2023. https://doi.org/10.1145/3558481.3591087.

[9] Zexun Wang. Proof-Carrying Agent Actions: Model-Agnostic Runtime Governance for Heterogeneous Agent Systems. arXiv:2606.04104v1, June 2026. https://arxiv.org/abs/2606.04104v1.

[10] Zihao Zheng, Baichuan Li, Junyi Yao, and Jiayu Long. Engineering Reliable Commit Gates for Agentic AI: Cost-Aware Verification Portfolios under Common-Mode Data Failures. arXiv:2609.10969v1, September 2026. https://arxiv.org/abs/2609.10969v1.

[11] Wassily Hoeffding. Probability Inequalities for Sums of Bounded Random Variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963. https://doi.org/10.1080/01621459.1963.10500830.

[12] NetworkX developers. NetworkX 3.6.1 release documentation and distributed source tests. https://networkx.org/documentation/stable/release/release_3.6.1.html.

[13] PyToolz contributors. Toolz API documentation and distributed source tests. https://toolz.readthedocs.io/en/stable/api.html.

[14] Jonathan E. Atkins, Erik G. Boman, and Bruce Hendrickson. A Spectral Algorithm for Seriation and the Consecutive Ones Problem. *SIAM Journal on Computing*, 28(1):297–310, 1998. https://doi.org/10.1137/S0097539795285771.

[15] Mark Herbster and Manfred K. Warmuth. Tracking the Best Expert. *Machine Learning*, 32:151–178, 1998. https://doi.org/10.1023/A:1007424614876.

[16] Sascha Geulen, Berthold Vöcking, and Melanie Winkler. Regret Minimization for Online Buffering Problems Using the Weighted Majority Algorithm. *Proceedings of COLT*, pages 132–143, 2010. https://www.learningtheory.org/colt2010/conference-website/papers/105geulen.pdf.

[17] Nicolò Cesa-Bianchi, Ofer Dekel, and Ohad Shamir. Online Learning with Switching Costs and Other Adaptive Adversaries. *Advances in Neural Information Processing Systems*, 26, 2013. https://papers.neurips.cc/paper/5151-online-learning-with-switching-costs-and-other-adaptive-adversaries.pdf.

[18] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *International Conference on Learning Representations*, 2024. https://arxiv.org/abs/2310.06770.

[19] OpenTelemetry contributors. Batch Processor documentation, official collector implementation. Accessed September 20, 2026. https://github.com/open-telemetry/opentelemetry-collector/tree/main/processor/batchprocessor.

[20] OpenAI contributors. tiktoken: BPE tokenization implementation and encoding registry. Accessed September 21, 2026; package/vocabulary unavailable in the v5 measurement runtime.

https://github.com/openai/tiktoken.

[21] SWE-bench contributors. The Harness: Docker-based patch evaluation reference. Accessed September 20, 2026.
https://www.swebench.com/SWE-bench/reference/harness/.

[22] B. Constantine, G. Forget, R. Geib, and R. Schrage. Framework for TCP Throughput Testing. RFC 6349, August 2011.
https://www.rfc-editor.org/rfc/rfc6349.html.

[23] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. arXiv:2405.15793, 2024.
https://arxiv.org/abs/2405.15793.

[24] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based Software Engineering Agents. arXiv:2407.01489, 2024.
https://arxiv.org/abs/2407.01489.

[25] Xingyao Wang et al. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. arXiv:2407.16741, 2024.
https://arxiv.org/abs/2407.16741.

[26] Tue Le, Minh V. T. Thai, Dung Nguyen Manh, Huy Phan Nhat, and Nghi D. Q. Bui. SWE-EVO: Benchmarking Coding Agents in Long-Horizon Software Evolution Scenarios. arXiv:2512.18470, revised 2026.
https://arxiv.org/abs/2512.18470.

[27] Qinghua Lu, Liming Zhu, Suyu Ma, Helen Power, and Robert Speight. Verification-Driven AI Engineering: Workflows and Reference Architecture. SSRN 6430979, 2026.
https://doi.org/10.2139/ssrn.6430979.

[28] Qinghua Lu, Ahmed E. Hassan, David Lo, and Thomas Zimmermann. Engineering Agentic Systems: Toward Verification-First AIware Engineering. *IEEE Software*, 43(5):22–29, 2026.
https://doi.org/10.1109/MS.2026.3697911.

[29] Ciprian Paduraru, Petru-Liviu Bouruc, and Alin Stefanescu. A Trace-Based Assurance Framework for Agentic AI Orchestration: Contracts, Testing, and Governance. arXiv:2603.18096, March 2026.
https://arxiv.org/abs/2603.18096.

A Optional Fixed-Share Analysis: Not the Deployment Recommendation

This section retains v3's former Theorem 4 and its assumptions. It is an optional comparator, not the practical selector recommended in v4. On the measured streams it loses to the deterministic priced-window rule in three regimes. We have not searched for a favorable adversarial stream after seeing those results, and make no claim that a tracking bound implies an empirical advantage.

A.1 Fixed-share weights with maximal coupling

Let a declared bound R cover each round's loss spread, $\max_i \ell_t(i) - \min_i \ell_t(i) \le R$. Reject a measurement exceeding it instead of clipping the cost. Define $z_{t,i} = (\ell_t(i) - \min_j \ell_t(j))/R$. Starting

at $p_1(i) = 1/K$, update

$$q_{t+1}(i) = \frac{p_t(i)\exp(-\eta z_{t,i})}{\sum_j p_t(j)\exp(-\eta z_{t,j})}, \tag{18}$$

$$p_{t+1}(i) = (1-\alpha)q_{t+1}(i) + \alpha/K, \quad 0 < \eta \le 1,\ 0 \le \alpha < 1. \tag{19}$$

Independent draws from each new distribution can waste migrations. Instead, conditional on $A_t = i$, retain i with probability $\min\{1, p_{t+1}(i)/p_t(i)\}$. If it is not retained, sample from the normalized positive residual $(p_{t+1} - p_t)_+$. This maximal coupling has marginal p_{t+1} and switch probability $\mathrm{TV}(p_t, p_{t+1})$. Established dependent-sampling methods motivate this choice [16]; we do not claim it as a new sampling algorithm.

Theorem 8 (Catalogue tracking with migration)**.** *Assume a frozen catalogue, a fixed exogenous sequence of full-information loss vectors, spread at most R, and $M_{ij} \le M$. For any comparator path $u_{1:T}$ with s changes and $0 < \alpha < 1$, the coupled policy satisfies*

$$\mathbb{E}[\mathcal{L}_T] - \sum_t \ell_t(u_t) \le R\left[\frac{\log K + s\log(K/\alpha) + (T-1-s)\log(1/(1-\alpha))}{\eta} + \frac{\eta T}{8}\right] + M(T-1)(\eta+\alpha). \tag{20}$$

For $\alpha = 0$ the stationary-comparator bound has numerator $\log K$ and applies only to $s = 0$.

Proof. The fixed-share update is exponential weighting over paths with an initial uniform prior, self-transition mass at least $1-\alpha$, and transition mass at least α/K. The negative log prior of a path with s switches is bounded by the numerator. The bounded exponential-moment inequality gives expected normalized service regret at most that numerator divided by η, plus $\eta T/8$ (Appendix D). Since losses are exogenous, the weights are independent of the sampled action path, and the coupling preserves each unconditional marginal. Exponential reweighting changes total variation by at most η; uniform mixing adds at most α. Thus the expected switch count is at most $(T-1)(\eta+\alpha)$. Multiplication by M and addition of service regret proves the bound. □

The bound also upper-bounds regret relative to the comparator's service *plus* nonnegative migration cost, but can be quite loose. It is not a per-run improvement guarantee. Choosing fixed $\eta = .5$ and $\alpha = .01$, as in our frozen experiment, need not make this bound sublinear with increasing T. Horizon-dependent tuning yields sublinear stationary bounds (Appendix D), without creating a guarantee against arbitrary reactive agents or newly introduced catalogue entries.

B Block-Backbone Compiler: Retained Ablation

The exact compiler's cubic cost makes frequent full recompilation inappropriate. For a supplied display order, divide the leaves into consecutive blocks of at most s leaves, and fix a compact b-ary macro-tree on those blocks. Optimize each block's microtree using its restricted completion waves and strict-epoch initialization cost. Replace each macro leaf by its optimized microtree. No evaluation waves enter this process.

Proposition 9 (Exactness in the block-backbone class)**.** *With display order, block boundaries and macro-tree fixed, independent exact optimization of the blocks minimizes the training objective over every layout in that restricted class. The dynamic-programming work is $O(bns^2)$ with $O(bs^2)$ reusable DP workspace, excluding order learning and wave preparation.*

Proof. Every macro node covers a fixed union of complete blocks, so its activation weight and packet price are constant across the class. All variable costs lie inside disjoint microtrees. The objective is a constant plus the sum of their independent objectives. Conditional exactness

therefore gives the minimizing member. There are at most $\lceil n/s \rceil$ subproblems, each of size at most s, yielding the stated DP bound. Reusing the subproblem arrays yields the workspace bound. □

For the reference implementation, M recorded waves and L total wave memberships add filtering and weight-preparation work of $O(\lceil n/s \rceil L + Mns)$, and storage for the input, output and local $M \times s$ arrays. This is not a linear-time claim for the entire pipeline. Dense affinity construction and order learning remain separate costs. No bounded approximation ratio against the unrestricted exact tree is asserted; held-out cost can deteriorate. The historical experiment froze $s = 32$ before measurement. Its negative result motivates excluding this compiler from the v5 practical path.

The original 18-workload v4 scaling data remain in the artifact. Block DP reduced fitting time relative to exact DP but lost to same-order balanced fixed8 in held-out bytes. Restricted-class exactness is a mathematical property, not an empirical recommendation.

C Information Separations Retained from Earlier Versions

These results concern the information available to a selector. They are not hardness results for all adaptive tree algorithms, and they do not imply that a complete hypergraph must be recovered: interval hit statistics are sufficient for a fixed order and objective.

C.1 Marginals do not determine a co-invalidation optimum

Consider four ordered leaves, binary trees, and a constant internal packet price $c > 0$. Let $\mathcal{D}_A$ choose $\{0, 1\}$ or $\{2, 3\}$ equiprobably and $\mathcal{D}_B$ choose $\{0, 3\}$ or $\{1, 2\}$ equiprobably. All leaf marginal activation probabilities equal $1/2$ in both distributions. The root and every three-leaf interval are always activated. The five tree shapes therefore fall into the following classes:

Ordered tree	J_A/c	J_B/c
$((0, 1), (2, 3))$	2	3
$(0, ((1, 2), 3))$, $((0, (1, 2)), 3)$	3	2.5
$(0, (1, (2, 3)))$, $(((0, 1), 2), 3)$	2.5	3

Optimal costs are $2c$ and $2.5c$. A deterministic marginal-only selector receives identical input and must choose the same class, incurring at least $c/2$ regret on one distribution. The last class is weakly dominated by the balanced tree. If a randomized selector chooses the balanced tree with probability q, its regrets are at least $(1 - q)c$ and $qc/2$. Their maximum is minimized at $q = 2/3$ and equals $c/3$, which is attainable for this instance. A fixed initialization price adds the same $3c$ to all binary shapes and does not alter the separation.

C.2 Joint invalidation does not determine progressive layout

Every epoch now invalidates all four checks, and execution order is $(0, 1, 2, 3)$ with singleton completion waves. In workload A, check 0 fails, so only its wave is executed. In workload B, check 3 fails, so all four checks execute. Initial invalidation is identical, even jointly.

Every binary tree has three internal nodes. Its cost in A is $(3+\text{depth}(0))c$, minimized at $4c$ by a root-0 comb. Its cost in B is $(3+\sum_i \text{depth}(i))c$, minimized at $11c$ by the balanced tree. The root-0 comb costs $12c$ in B, whereas the balanced tree costs $5c$ in A. The remaining shapes are weakly dominated. An invalidation-only randomized selector using a root-0 comb with probability p incurs regrets at least $(1 - p)c$ and pc, whose maximum is at least $c/2$. A deterministic selector incurs at least c. The example proves a separation between stale-set information and schedule-induced completion information; it does not prohibit a selector that observes both.

D Cost, Statistical, and Online Proof Details

D.1 Exact serialization accounting

On the first refresh in a strict epoch, every internal node emits one packet. Within a fixed tree and registry, the cache uses monotone descendant-generation stamps. A subsequent wave emits a node's packet exactly when at least one final record in its semantic coverage changed. Multiple completed descendants in one wave still induce one packet at that node. Summing initialization and wave contributions yields Equation (4). After migration, a new kernel remaps canonical IDs and the old ordinal cache is not reused; the entire cold rebuild is charged in Equation (12).

This argument concerns logical and actual serialized packet lengths. The separate v4 study measures controlled application-level export time, not production-network throughput. It does not include raw log streams, TLS framing, operating-system scheduling, or Python traversal costs. The reference kernel validates and traverses trees and is not claimed to have logarithmic total runtime. Small contexts do not imply small total system state.

D.2 Full-refresh lower bound and global negative control

For an n-leaf tree with m internal nodes, the number of edges is $n+m-1$, so $\sum_v d_v = n+m-1$. Therefore

$$W(L) = S(n-1) + (H+S)m \geq S(n-1) + (H+S)\left\lceil \frac{n-1}{b-1} \right\rceil. \tag{21}$$

The inequality follows from $n-1 = \sum_v (d_v - 1) \leq m(b-1)$. At $n = 64, b = 8$, a complete eight-child construction attains the bound with $m = 9$ and $W = 27,648$ bytes. Thus all selected global-control layouts can tie an analytically minimal full refresh. Strict initialization followed by one global completion wave costs $2W$. This explains the exact negative control without assuming every balanced tree at every n attains the lower bound.

D.3 Uniform statistical control at a fixed order

For one activation wave per episode and a fixed registry order, let $M_n = n(n+1)/2$ and

$$\epsilon = \sqrt{\log(2M_n/\delta)/(2m)}.$$

The interval hit indicators are bounded independent variables *across* independent episodes, though checks within an episode may be arbitrarily dependent. Hoeffding plus a union bound gives simultaneous interval-frequency error at most ϵ. For every feasible tree, $|\widehat{J}(T) - J(T)| \leq W(T)\epsilon$. Inserting the empirical minimizer gives regret at most $[W(\widehat{T}) + W(T^*)]\epsilon \leq 2(n-1)C\epsilon$. This is the original HACT bound, retained for attribution. It is not automatically valid uniformly over arbitrary data-selected permutations; the separate held-out catalogue bound in Proposition 5 addresses that selection step.

For strict epochs with at most q completion waves, a loose range is $B = (q+1)(n-1)C$. Under the evaluated sequential eight-record flush rule with at most one final record per obligation, $q \leq \lceil n/8 \rceil$. A narrower observed range is not automatically a proved bound for an unobserved deployment distribution.

D.4 Maximal coupling and its switch count

Let p, p' be consecutive distributions. The probability mass that remains at index i under the coupling is $\min(p_i, p'_i)$. The remaining mass is distributed as $(p'-p)_+$, so the new marginal is exactly p' and

$$\mathbb{P}(A_{t+1} \neq A_t) = 1 - \sum_i \min(p_i, p'_i) = \mathrm{TV}(p, p').$$

For exponential reweighting by $a_i = \exp(-\eta z_i)$ with $z_i \in [0,1]$, the normalizer is at most one, hence $q_i \geq e^{-\eta} p_i$. Thus $\sum_i \min(p_i, q_i) \geq e^{-\eta}$ and $\mathrm{TV}(p,q) \leq 1 - e^{-\eta} \leq \eta$. Mixing q with the uniform distribution by weight α changes total variation by at most α. The triangle inequality gives the switch bound in Theorem 8. Zero old mass is never the probability of a selected previous action in exact arithmetic; the implementation rejects inconsistent checkpoints.

D.5 Fixed-share path argument

A path $u_{1:T}$ with s changes has prior mass at least

$$P(u_{1:T}) \geq K^{-1}(\alpha/K)^s(1-\alpha)^{T-1-s}.$$

The share update is the forward recursion for exponential weighting over such paths. If $Z_t = \sum_i p_t(i)\exp(-\eta z_{t,i})$, then the bounded exponential-moment inequality implies

$$\log Z_t \leq -\eta\langle p_t, z_t\rangle + \eta^2/8.$$

On the other hand, the cumulative mixture likelihood is at least the comparator path's prior times its exponential loss. Combining the inequalities yields

$$\sum_t \langle p_t, z_t\rangle - \sum_t z_{t,u_t} \leq -\log P(u_{1:T})/\eta + \eta T/8.$$

Centering losses by their per-round minimum cancels in regret, and multiplying by R restores byte units. For a fixed exogenous cost sequence, each p_t is determined by past full-information observations independently of the sampled action sequence; maximal coupling preserves its marginal. Therefore expected sampled service equals the left-side weighted service. Adding at most M per expected switch proves Equation (20).

The result does not cover a cost sequence chosen in reaction to the coupled hidden action history. It also does not cover agents changing future task choices because a layout changed the text they read. These limitations are why the paper distinguishes a verification-plane cost experiment from an end-to-end learning claim.

D.6 Horizon dependence and break-even interpretation

For $K \geq 2$, $s = 0$, and $\alpha = 0$, write $B_T = RT/8 + M(T-1)$. The bound is $R\log K/\eta + \eta B_T$. When $\sqrt{R\log K/B_T} \leq 1$, choosing that learning rate gives at most $2\sqrt{R\log K\, B_T}$ expected regret, which is sublinear for fixed R, M, K. This is an established style of switching-cost bound, not a new rate. With a known sparse switch budget, selecting a horizon-dependent share similarly prices tracking complexity. Our fixed-parameter experiments do not invoke such a tuned asymptotic guarantee.

A single empirical switch with estimated future saving d and anticipated remaining horizon h satisfies the familiar break-even inequality $hd > M_{ij}$. This is arithmetic, not a drift theorem: savings may reverse before h events. The included window rule uses its past 64 events as that estimate and is deliberately reported as a heuristic. All choices, including greedy choices that ignore migration when deciding, are charged their actual transition price during evaluation.

E Experimental Reproduction and Threats to Validity

E.1 Software and data identities

The archived v3 runs used Python 3.13.5 on Linux x86_64; exact installed package and platform records accompany emitted reports. Compilers were measured with `OPENBLAS_NUM_THREADS=1`.

No accelerator is required. Vendored upstream sources and tests are retained with their notices. Every fresh mutation is checked against its archived base-source hash, replacement span, and resulting source hash. This binds a result to a specific intervention, not a natural historical bug distribution.

For the archived v3 record, there are three named output categories: `historical/results_v2` contains previous observations; `results/v3` contains v3's generated layouts, raw waves, online paths, fresh subprocess reports, and harness records; `paper/generated` contains tables regenerated from those files. Running reproduction in a fresh destination avoids overwriting the released record. The release manifest identifies the exact packaged files. The final extracted archive is tested again before delivery. New v4 transfers, compiler traces, contexts, summaries and audits are stored separately in `results/v4`; they do not change the number of earlier checker or harness executions. V5 adds `results/v5` for the practical catalogue, archived-source re-aggregation, conditional deployment accounting and failed BPE attempts. Neither a new layout nor a reproduced experiment is counted as another repaired candidate.

E.2 Costs not hidden by replay

Ordering measurements include strict initialization; online traces explicitly use persistent-state incremental service plus cold switch prices. These are different lifecycles. Source replay uses a fixed previous ranker; fresh paired executions hold that ranker constant. The I/O harness does not compare different model-generated solutions: it sends identical scripted patch content to the same real runtime under different gates. Its model endpoint produces no provider token-usage field, so absent usage remains null in the records. Prompt byte lengths are measured directly and never converted to tokens by a heuristic multiplier.

Timing rows report outer-wrapper or active-episode boundaries explicitly. The wrapper's internal total ends just before final report and packet-file writes; the fresh study additionally records the outer call duration including those writes. Fit, contract approval, and baseline preparation are setup costs, not silently amortized into a claimed end-to-end speedup. With only three harness repeats, ranges are more honest than a precise broad-population confidence claim.

E.3 Test adequacy, nondeterminism, and I/O

The fixture performs real disk and loopback I/O but is intentionally controlled. It is not a test of internet reliability, production SQLite contention, nondeterministic service responses, or all flaky-test policies. When a collected check is skipped, missing, or inconsistent, it is not accepted as a pass. Two contradictory final records in one epoch trigger conflict handling, and an operator must establish a new epoch rather than cherry-pick the favorable record. Actual source tests can themselves be incomplete; agreement with their oracle is only agreement with the declared acceptance contract.

F Artifact Usage and Integration Contract

A reviewed operator baseline is initialized outside candidate write scopes:

```
python -m ichact init --project /work/trusted-baseline \
  --contract /work/contracts/approved.json \
  --output /work/reports/baseline --test-path tests

python -m ichact verify --project /work/candidate \
  --contract /work/contracts/approved.json \
  --output /work/reports/candidate-001 \
  --layout /work/contracts/layout.json
```

A layout must match the full semantic registry. The optional execution-order file is separate and must also be an exact permutation, never a subset. Legacy schema-1 contracts must be reviewed and reinitialized; the software does not silently reinterpret them.

The Future Code gate invokes the installed module or a pinned absolute bridge path outside the candidate:

```
python -m ichact.future_bridge \
  --contract /work/contracts/approved.json \
  --reports /work/verification-reports \
  --layout /work/contracts/layout.json
```

The working directory is the candidate snapshot. Exit 0 means complete current pass; nonzero means rejection or unknown. This CLI does not itself deploy or merge. The original harness performs its own staged-source fingerprint and integration checks. For hostile code, use an outer container/VM and do not give it access to secrets or trusted contract storage.

The artifact contains an actual scripted harness campaign, a separate live-endpoint configuration guide, deterministic tests, an independent audit, and an isolated reproduction launcher. The live guide is a deployment recipe, not evidence that paid inference or hosted evaluation was performed.